

企业级 IM 项目向开源 AI 产品升级长期计划书：V24-V40

一、产品最终设想

当前项目已经规划为：

```text id="2m4dcv" 基于 Go-Zero 的企业级分布式 IM 与实时事件通知系统

后续可以继续升级为：

```text id="quvwed"  
私有化 AI Agent 工作台 / 局域网 AI 任务协作平台

产品定位：

```text id="ssb04w" 一个可以私有化部署的 AI 工作台，支持团队成员通过聊天界面提交任务，AI Agent 自动拆解、检索知识库、调用工具、执行流程，并通过实时消息系统推送任务进度、执行日志和结果。

产品关键词：

```text id="kqpa0t"  
私有化部署
局域网可用
AI Agent
RAG 知识库
工具调用
任务状态实时推送
团队协作
代码分析
文档生成
可观测执行过程

最终项目名称可以考虑：

```text id="09agkm" DecisionOS AgentDesk LocalAgentHub TeamAgent FlowIM AI

其中你之前提到过 DECISIONOS，这个名字可以保留，但建议产品定位不要太大，第一阶段可以叫：

```text id="s6y06d"  
DecisionOS：私有化 AI 任务协作工作台

二、从 IM 系统到 AI 产品的核心转变

原系统核心是：

```text id="nxul00" 人和人之间实时通信

升级后核心变成：

```text id="3op6q4"  
人、AI Agent、工具、任务之间实时协作

原 IM 系统中的能力可以复用为：

```text id="a0jply" WebSocket 长连接 → AI 任务进度实时推送 会话系统 → AI 任务会话 消息系统 → 用户消息、Agent 消息、工具日志 群聊系统 → 多人协作任务空间 文件消息 → 上传文档、代码包、数据文件 离线补偿 → 用户回来后同步任务执行结果 管理后台 → 管理模型、工具、知识库、用户权限 监控系统 → 观察 Agent 执行耗时、失败率、Token 消耗

所以不是重新开一个 AI 项目，而是把 IM 升级为：

```text id="1605ye"  
实时协作底座 + AI 任务执行引擎

三、长期版本规划

完整升级路线：

```text id="n6bfht" V24：AI 产品定位与最小闭环设计 V25：AI 会话与任务中心 V26：Agent 任务执行引擎 V27：实时执行过程推送 V28：RAG 知识库模块 V29：文件、网页、代码仓库接入 V30：工具调用与插件系统 V31：MCP / Tool Server 兼容层 V32：工作流编排与多 Agent 协作 V33：代码仓库分析与自动修改助手 V34：AI Ops / 日志诊断助手 V35：团队协作空间与权限系统 V36：模型配置、Token 成本与执行安全 V37：本地化部署与一键安装 V38：开源项目包装与社区增长 V39：模板市场与插件生态 V40：商业化与企业版能力

其中：

```text id="x0pk37"  
V24-V27：AI 产品最小闭环
V28-V31：AI 能力增强
V32-V34：形成差异化场景
V35-V37：变成可用产品
V38-V40：冲击开源项目影响力

V24: AI 产品定位与最小闭环设计

目标

先确定产品不是泛泛的 AI 聊天工具，而是一个“任务型 AI 工作台”。

产品最小闭环

最小闭环只做一件事：

```text id="mr7krq" 用户提交一个任务，AI 分析任务，分步骤执行，并实时展示执行过程和最终结果。

示例任务：

```text id="hmn5y"  
帮我分析这个 Go 项目的架构，并指出可以优化的地方。

执行过程：

```text id="0mchta" 用户上传项目压缩包 ↓ AI 创建任务 ↓ 解析目录结构 ↓ 分析核心模块 ↓ 生成项目架构总结 ↓ 生成优化建议 ↓ 生成面试追问 ↓ 实时推送每一步状态

## 产品第一阶段不要做

```text id="1qv86t"  
不要做通用 ChatGPT
不要做复杂多 Agent
不要做自动爬全网
不要做复杂商业 CRM
不要做浏览器自动操作
不要做太多模型适配

V24 产出

```text id="j4zg3j" PRODUCT\_POSITIONING.md MVP\_SCOPE.md USER\_STORY.md ROADMAP.md

## Codex 任务

```text id="zxsus1"  
请基于当前 IM 实时消息系统，设计一个名为 DecisionOS 的私有化 AI 任务协作工作台 MVP。要求输出产品定位、目标用户、核心使用场景、MVP 功能范围、用户故事、页面结构和后端服务规划。不要写代码，只生成 docs/product/ 下的设计文档。

V25: AI 会话与任务中心

目标

把普通聊天会话升级为 AI 任务会话。

新增概念

```text id="h4zzim" AI Session: 一次 AI 会话 AI Task: 一次可执行任务 Task Message: 任务相关消息 Task Event: 任务执行过程事件

## 新增页面

```text id="ne3dms"

AI 工作台首页

任务列表

任务详情页

AI 会话页

任务执行日志页

数据表

```text id="wf44ez" ai\_sessions - id - user\_id - title - status - created\_at - updated\_at

ai\_tasks - id - session\_id - user\_id - task\_type - input - status - result - error - created\_at - updated\_at

ai\_task\_events - id - task\_id - event\_type - content - created\_at

## 任务状态

```text id="b02f5n"

created

planning

running

waiting_confirm

succeeded

failed

cancelled

验收标准

```text id="pf9l7b" 1. 用户可以创建 AI 会话 2. 用户可以提交 AI 任务 3. 页面能展示任务列表 4. 页面能展示任务详情 5. 任务状态可以更新 6. 任务事件能存储到数据库

```
Codex 任务
```

```
```text id="hp3rv2"
```

请在当前 IM 系统中新增 AI Session 和 AI Task 模块。要求新增 ai_sessions、ai_tasks、ai_task_events 表，实现创建 AI 会话、提交任务、查询任务列表、查询任务详情、更新任务状态和记录任务事件的接口。前端新增 AI 工作台、任务列表和任务详情页面。

V26: Agent 任务执行引擎

目标

实现一个最小可用的 Agent 执行引擎。

它不需要一开始很复杂，只需要支持：

```
```text id="lt6bsq" 任务理解 任务拆解 步骤执行 结果总结 失败记录
```

```
Agent 执行流程
```

```
```text id="ippv1a"
```

用户提交任务

↓

Planner 生成执行计划

↓

Executor 按步骤执行

↓

每一步生成事件

↓

Reviewer 汇总结果

↓

保存最终输出

最小 Agent 组件

```
```text id="bqz3r5" Planner: 拆解任务 Executor: 执行步骤 Reviewer: 总结结果 Memory: 读取历史任务  
上下文 EventPublisher: 推送任务事件
```

```
任务事件类型
```

```
```text id="iifncn"
```

TASK_CREATED

PLAN_STARTED

PLAN_FINISHED

STEP_STARTED

```
TOOL_CALL_STARTED
TOOL_CALL_FINISHED
STEP_FINISHED
TASK_SUCCEEDED
TASK_FAILED
```

验收标准

```text id="ngkm2w" 1. 提交任务后能生成执行计划 2. 执行计划至少包含 3-5 个步骤 3. 每个步骤执行时能记录事件 4. 失败时能记录错误原因 5. 成功后能生成最终总结

## Codex 任务

```text id="0bm9iu"

请为 DecisionOS 新增最小 Agent 执行引擎。要求实现 Planner、Executor、Reviewer 和 EventPublisher 四个核心组件。用户提交任务后，Planner 生成步骤计划，Executor 逐步执行，Reviewer 生成最终结果，EventPublisher 将每一步状态写入 ai_task_events 表，并通过现有 IM WebSocket 事件通道推送给前端。

V27：实时执行过程推送

目标

把 IM 的实时消息能力真正复用到 AI 产品中。

用户不应该只看到一个最终结果，而应该看到：

```text id="td3k4u" AI 正在分析什么 调用了什么工具 当前执行到哪一步 是否失败 是否需要用户确认

## 前端表现

任务详情页展示：

```text id="1wbxjr"

任务状态时间线
实时日志流
当前执行步骤
工具调用结果
最终报告

WebSocket 消息类型

```text id="1bgxw" task\_event agent\_event tool\_event ai\_stream

## 示例事件

```
```json id="4fqhif"
{
  "type": "task_event",
  "taskId": "task_001",
  "eventType": "STEP_STARTED",
  "content": "正在分析项目目录结构"
}
```

验收标准

```text id="3uybok" 1. AI 任务执行过程中前端能实时看到事件 2. 刷新页面后可以从数据库恢复历史事件 3. 用户离线后重新登录可以同步任务结果 4. 任务失败时能显示失败原因

## Codex 任务

```text id="ib7cyu"  
请将 AI Task 的执行事件接入现有 IM WebSocket 实时消息系统。要求新增 task_event、agent_event、tool_event、ai_stream 四类消息，任务执行过程中实时推送到前端任务详情页。前端需要支持实时事件流展示，并在刷新页面后从 ai_task_events 表恢复历史事件。

V28：RAG 知识库模块

目标

让 AI 不只是聊天，而是能基于用户自己的资料回答问题。

知识库能力

```text id="43qwxl" 创建知识库 上传文档 文档切分 向量化 检索 引用片段展示 基于知识库回答

## 数据表

```
```text id="8ef1tx"
knowledge_bases
documents
document_chunks
embedding_records
```

技术选型

第一阶段可以选：

```text id="3q2cwk" PostgreSQL + pgvector

如果想降低复杂度，也可以先用：

```text id="t15tjm"  
SQLite / MySQL 存 chunk + 简单关键词检索

MVP 阶段不一定必须接复杂向量库。

知识库使用场景

```text id="o67osm" 上传项目 README 上传接口文档 上传论文资料 上传公司内部文档 上传个人笔记

## 验收标准

```text id="mirxyw"  
1. 用户可以创建知识库
2. 可以上传文档
3. 文档可以切分成 chunk
4. 用户提问时能检索相关 chunk
5. AI 回答中能引用来源

Codex 任务

```text id="v7tm4j" 请为 DecisionOS 新增 RAG 知识库模块。要求支持创建知识库、上传文档、文档切分、chunk 存储、检索相关片段，并在 AI Task 执行时可以选择知识库作为上下文。第一版可以先实现关键词检索或简单向量检索，预留 pgvector 接口。

---

# V29: 文件、网页、代码仓库接入

## 目标

让用户可以把真实资料接入产品。

## 接入类型

```text id="4k4xxj"  
普通文件
PDF / Word / Markdown
网页 URL
GitHub / Gitee 代码仓库
本地代码压缩包
日志文件

重点场景

第一阶段重点做：

```text id="nqoy3o" 代码仓库压缩包分析 Markdown / README 分析 日志文件分析

不要一开始做复杂爬虫。

## 代码仓库分析流程

```text id="6xb4br"

上传 zip

↓

解压到隔离目录

↓

扫描目录结构

↓

识别语言和框架

↓

提取 README、配置文件、核心代码

↓

生成项目结构摘要

↓

进入 Agent 分析任务

验收标准

```text id="ltx0x" 1. 用户可以上传代码 zip 2. 系统可以解析目录结构 3. 系统可以生成项目文件树 4. AI 可以基于项目文件生成分析报告 5. 文件解析失败时有错误提示

## Codex 任务

```text id="jzui8k"

请为 DecisionOS 新增文件与代码仓库接入能力。要求支持上传 zip 格式代码仓库，服务端解压到隔离目录，扫描目录结构，识别 README、go.mod、pom.xml、package.json、Dockerfile 等关键文件，并生成项目结构摘要供 AI Task 使用。

V30：工具调用与插件系统

目标

让 Agent 不只是生成文本，而是可以调用工具完成任务。

第一批工具

```text id="02sevv" read\_file: 读取文件 list\_files: 列出目录 search\_code: 搜索代码 run\_command: 执行安全命令 query\_database: 查询数据库 http\_request: 调用 HTTP 接口 generate\_doc: 生成文档

## 工具调用记录

```
```text id="b9hlwb"
tool_calls
- id
- task_id
- tool_name
- input
- output
- status
- error
- created_at
```

安全限制

```text id="v7hl7s" 命令白名单 工作目录隔离 执行超时 输出长度限制 禁止危险命令 记录所有工具调用

## 验收标准

```
```text id="xgum1i"
1. Agent 可以调用 list_files / read_file
2. 工具调用过程会记录到数据库
3. 工具调用事件能实时推送
4. 危险命令会被拒绝
5. 工具失败后 Agent 能继续总结错误
```

Codex 任务

```text id="gmno0vo" 请为 DecisionOS 新增 Tool Calling 系统。要求实现工具注册表，第一批工具包括 list\_files、read\_file、search\_code、run\_command。所有工具调用需要写入 tool\_calls 表，并通过 task\_event 实时推送。run\_command 必须限制工作目录、命令白名单和执行超时，避免危险命令。

---

# V31: MCP / Tool Server 兼容层

## 目标

让产品具有开源生态潜力。

MCP 可以作为后续方向，先做兼容设计，不要一开始重度实现。

```
设计目标
```

```
```text id="0gq0hd"
```

内部工具系统

↓

Tool Adapter

↓

MCP Server / 外部工具

第一阶段

只做：

```
```text id="gdsfrx" MCP 配置文件 外部工具注册抽象 工具协议适配接口
```

```
验收标准
```

```
```text id="r4l7ne"
```

1. 工具系统支持本地工具和外部工具两种类型
2. 可以通过配置注册一个外部工具
3. Agent 调用时不关心工具来源
4. 文档说明未来如何接 MCP

Codex 任务

```
```text id="qlnjwv" 请为 Tool Calling 系统设计 MCP / 外部工具兼容层。第一版不需要完整实现 MCP 协议，但需要抽象 ToolProvider、LocalToolProvider、ExternalToolProvider，并提供工具配置文件格式，使 Agent 调用工具时不关心工具来源。
```

```

```

```
V32: workflow编排与多 Agent 协作
```

```
目标
```

让产品从单 Agent 任务升级为可配置工作流。

```
工作流概念
```

```
```text id="y63c3b"
```

节点

边

条件分支

人工确认

工具调用

Agent 节点
输出节点

示例 workflow

```text id="qvbb5l" 代码仓库分析 workflow: 1. 扫描目录 2. 识别技术栈 3. 分析核心模块 4. 找出风险点 5. 生成优化建议 6. 生成报告

```
多 Agent 角色

```text id="av4752"
Planner Agent
Coder Agent
Reviewer Agent
Ops Agent
Doc Agent
```

前端页面

```text id="7oh3db" workflow 模板列表 workflow 执行详情 节点执行状态 执行日志

```
验收标准

```text id="715qbo"
1. 可以定义一个固定 workflow 模板
2. Agent 按 workflow 节点顺序执行
3. 每个节点状态实时展示
4. 失败节点可重试
5. workflow 执行结果可保存
```

Codex 任务

```text id="b8tihm" 请为 DecisionOS 新增 workflow 执行引擎的 MVP。第一版只需要支持固定模板 workflow，包括节点定义、顺序执行、节点状态记录、失败重试和执行事件推送。请实现一个“代码仓库分析 workflow”模板，包含目录扫描、技术栈识别、核心模块分析、风险分析和报告生成节点。

```

V33: 代码仓库分析与自动修改助手

目标

形成第一个真正有吸引力的开源场景。

用户上传代码仓库后，AI 可以：
```

```
```text id="7ql84g"
```

分析项目结构
解释核心模块
找出坏味道
生成优化建议
定位潜在 bug
生成 patch
执行测试
输出 diff

第一阶段不要自动提交代码

只做：

```
```text id="xcaf32" 生成建议 生成 diff 用户确认后应用 patch
```

```
核心流程
```

```
```text id="3gax3k"
```

上传代码仓库
↓
扫描项目结构
↓
AI 分析问题
↓
生成修改建议
↓
生成 patch
↓
用户确认
↓
应用 patch
↓
运行测试
↓
输出结果

需要的工具

```
```text id="e6d92n" list_files read_file search_code write_file apply_patch run_test git_diff rollback
```

```
验收标准
```

```
```text id="th2jwl"
```

1. 能分析 Go / Java / Python 项目结构
2. 能生成优化建议

- 3. 能生成 diff
- 4. 用户确认后才能修改文件
- 5. 修改后能展示 git diff
- 6. 失败时可以 rollback

Codex 任务

```text id="6q0fqv" 请为 DecisionOS 新增代码仓库分析与修改助手 MVP。要求支持上传代码仓库、扫描项目结构、生成优化建议、生成 patch、用户确认后应用 patch、展示 git diff，并提供 rollback 能力。所有文件修改必须记录到 task\_event 和 tool\_calls 中。

```

V34: AI Ops / 日志诊断助手

目标

形成第二个有价值场景：运维故障诊断。

这个方向和你前面的 IM 系统、监控系统结合很好。

功能

```text id="tzqemj"
上传日志
选择服务
AI 分析异常
关联指标
生成故障原因
生成排查步骤
生成修复建议
```

示例

```text id="cbekhw" 用户上传 gateway.log AI 发现 Redis 连接超时 结合消息发送失败率上升 判断是 Redis 不稳定导致在线路由查询失败 输出排查建议

```
验收标准

```text id="5aocax"
1. 可以上传日志文件
2. 可以按 traceId 查询相关日志
3. AI 可以总结异常类型
4. AI 可以生成排查步骤
5. AI 可以输出修复建议
```

Codex 任务

```text id="89rycy" 请为 DecisionOS 新增 AI Ops 日志诊断助手。要求支持上传日志文件、按 traceId 检索日志、分析错误堆栈、总结异常原因，并生成排查步骤和修复建议。该功能需要复用现有 task\_event 实时展示诊断过程。

---

# V35: 团队协作空间与权限系统

## 目标

让产品从个人工具升级为团队产品。

## 概念

```text id="bkywyr"

Workspace

Project

Member

Role

Permission

角色

```text id="1o8z3m" Owner Admin Member Viewer

## 权限

```text id="m8epgo"

谁可以上传文件

谁可以创建任务

谁可以查看任务

谁可以执行工具

谁可以管理知识库

谁可以配置模型

验收标准

```text id="i88r5d" 1. 用户可以创建 Workspace 2. 可以邀请成员 3. 不同成员有不同权限 4. 任务、知识库、代码仓库归属于 Workspace 5. 权限不足时拒绝访问

## Codex 任务

```text id="ifx0qk"

请为 DecisionOS 新增 Workspace 团队空间和 RBAC 权限系统。要求支持创建工作空间、邀请成员、角色

管理，并将 AI Task、Knowledge Base、Uploaded Files、Tool Calls 归属于 Workspace。不同角色拥有不同操作权限。

V36：模型配置、Token 成本与执行安全

目标

让产品具备真实使用的基础治理能力。

模型配置

支持：

``text id="xtms2i" OpenAI-compatible API 本地模型 API 不同模型配置 默认模型 备用模型

Token 成本统计

记录：

``text id="zpsb7u"
prompt_tokens
completion_tokens
total_tokens
estimated_cost
task_id
user_id
workspace_id

安全策略

``text id="f1lmcz" 工具权限控制 命令白名单 文件访问沙箱 敏感信息脱敏 人工确认节点 任务超时 最大 Token 限制

验收标准

- ``text id="51encg"
1. 管理员可以配置模型
 2. 每次调用记录 Token 用量
 3. 可以查看任务成本
 4. 高风险工具调用需要用户确认
 5. 命令执行受到沙箱限制

Codex 任务

```text id="l2izbq" 请为 DecisionOS 新增模型配置、Token 用量统计和执行安全策略。要求支持 OpenAI-compatible 模型配置，记录每次 LLM 调用的 token 用量和预估成本；工具调用需要权限控制，高风险操作需要人工确认，命令执行必须限制在沙箱目录内。

---

# V37: 本地化部署与一键安装

## 目标

让项目具备开源传播能力。

开源项目要想获得用户，必须安装简单。

## 安装方式

```text id="p6jz9s"

docker compose up -d

或者：

```text id="17k7fn" curl -fsSL xxx/install.sh | bash

## 支持模式

```text id="znid31"

单机模式

局域网模式

开发模式

生产模式

配置文件

```text id="cn34ed" .env config.yaml models.yaml tools.yaml

## 验收标准

```text id="6dnuiif"

1. 新用户 10 分钟内能启动项目
2. README 有清晰安装步骤
3. 启动后自动初始化数据库
4. 有默认管理员账号

- 5. 可以配置模型 API Key
- 6. 局域网其他设备可以访问

Codex 任务

```text id="ovmd02" 请为 DecisionOS 设计一键本地化部署方案。要求提供 docker-compose.yml、.env.example、config.yaml、models.yaml、tools.yaml、init.sql 和 install.sh。目标是新用户 10 分钟内可以在本机或局域网启动系统，并通过浏览器访问前端。

```

V38: 开源项目包装与社区增长

目标

让项目具备冲击 GitHub Stars 的基础。

开源项目必须补齐

```text id="rmqd2f"
README
英文 README
中文 README
项目 Logo
演示 GIF
在线截图
架构图
快速开始
Roadmap
贡献指南
Issue 模板
PR 模板
License
Docker 快速部署
Demo 视频
```

README 首页结构

```text id="5xpkyv" 1. 项目一句话介绍 2. Demo GIF 3. 核心特性 4. 为什么需要它 5. 快速开始 6. 架构图 7. 使用场景 8. 截图 9. Roadmap 10. Star History 11. Contributing

```
项目卖点

不要写：
```

```
```text id="d7h4jy"
一个 AI 平台
```

要写：

```
```text id="206iew" 一个可私有化部署的 AI Agent 工作台，通过实时事件流展示 Agent 执行过程，支持知识库、工具调用、代码仓库分析和团队协作。
```

```
验收标准
```

```
```text id="6cnafr"
```

1. 陌生用户看 README 30 秒知道项目做什么
2. 5 分钟看懂架构
3. 10 分钟能跑起来
4. 有动图展示
5. 有明确 Roadmap
6. 有适合传播的项目截图

Codex 任务

```
```text id="k78bnb" 请为 DecisionOS 生成开源项目包装文档，包括中英文 README、Roadmap、Contributing、Issue 模板、PR 模板、License 建议、项目截图清单、Demo GIF 脚本和首页卖点文案。目标是让陌生开发者 30 秒理解项目价值，10 分钟完成本地启动。
```

```

```

```
V39: 模板市场与插件生态
```

```
目标
```

让项目不只是单一工具，而是可扩展平台。

```
模板类型
```

```
```text id="jbtfdp"
```

代码仓库分析模板
日志诊断模板
论文总结模板
需求分析模板
日报周报生成模板
接口文档生成模板
测试用例生成模板

插件类型

```
```text id="0h5rxt" 工具插件 模型插件 知识库解析插件 通知插件 代码执行插件
```

## ## 验收标准

```text id="h0ks6t"

1. 用户可以选择任务模板
2. 模板可以定义输入字段
3. 模板可以绑定工作流
4. 插件可以通过配置注册
5. 社区可以贡献模板

Codex 任务

```text id="y6ftth" 请为 DecisionOS 设计模板和插件系统。要求支持任务模板定义、输入参数配置、工作流绑定和插件注册。第一版实现本地模板目录，用户可以选择“代码仓库分析”“日志诊断”“文档总结”等模板创建 AI Task。

---

## # V40: 商业化与企业版能力

### ## 目标

长期设想，不是当前阶段立即实现。

### ## 企业版能力

```text id="l3oewj"

多租户

SSO 登录

企业知识库

审计日志

权限审批

私有模型接入

LDAP / 企业微信 / 飞书集成

数据备份

高可用部署

商业模式设想

```text id="pg05l4" 开源社区版 专业版 企业私有化部署版 插件市场 咨询部署服务

### ## 当前不做

```text id="21z7i9"

不要现在做收费系统

不要现在做复杂多租户

不要现在做企业微信集成
不要现在做高可用集群

Codex 任务

```text id="9ywq8u" 请为 DecisionOS 输出企业版长期规划文档，包括多租户、SSO、审计日志、企业知识库、权限审批、私有模型接入和高可用部署等能力。该阶段只做规划，不做代码实现。

---

# 四、真正可能获得 GitHub Star 的最小产品版本

不要等 V40 全部完成才开源。

真正适合开源发布的第一个版本应该是：

```text id="uwg25o"  
DecisionOS v0.1: 私有化 AI 任务执行工作台

v0.1 必须包含：

```text id="nvtow6" 1. Docker Compose 一键启动 2. Web 前端 3. 用户登录 4. AI 任务提交 5. Agent 执行计划 6. 实时任务事件流 7. 文件上传 8. 代码仓库 zip 分析 9. RAG 简化知识库 10. 工具调用记录 11. 任务结果报告 12. 中英文 README 13. Demo GIF

v0.1 不需要包含：

```text id="f1sa9i"  
复杂多 Agent
完整 MCP
朋友圈
红包
完整 IM 社交体系
复杂权限系统
商业化
多租户

五、最推荐的实际实现顺序

第一阶段：IM 底座完成

```text id="4tcwst" V1-V11 V14 V16-V18 V21-V23

目标：

```text id="ofmsnr"

先得到一个可局域网使用的完整 IM 系统。

第二阶段：AI MVP

```text id="hbvqtv" V24：产品定位 V25：AI 会话与任务中心 V26：Agent 执行引擎 V27：实时执行过程推送 V29：文件和代码仓库接入 V30：基础工具调用

目标：

```text id="80c2iz"

用户可以上传代码仓库，AI 实时分析并生成报告。

第三阶段：知识库与 workflow

```text id="z56wmn" V28：RAG 知识库 V32：workflow 编排 V33：代码仓库分析与自动修改助手 V34：AI Ops 日志诊断助手

目标：

```text id="97ojjp"

形成两个清晰场景：代码分析助手 + 日志诊断助手。

第四阶段：开源包装

```text id="utlyc6" V37：一键安装 V38：开源项目包装 V39：模板插件生态

目标：

```text id="9yim7j"

让陌生人愿意安装、试用、Star。

第五阶段：商业化设想

```text id="gh8f3b" V35：团队空间 V36：模型成本与安全 V40：企业版规划

目标：

```
```text id="uynrd8"
从个人开源工具升级为团队产品。
```

六、项目最终架构设想

```
```text id="w1fn1g" 前端层： - Web Client - Admin Console - AI Workspace - PWA / Desktop Shell
```

网关层： - API Gateway - WebSocket Gateway

IM 实时通信层： - Message Service - Conversation Service - Contact Service - Group Service - Event Push Service

AI 任务层： - AI Session Service - AI Task Service - Agent Runtime - Workflow Engine - Tool Service - Knowledge Base Service

数据与基础设施： - MySQL - Redis - Kafka - MinIO - PostgreSQL / pgvector - Prometheus - Grafana - Docker Compose

开放扩展层： - Tool Plugin - MCP Adapter - Template System - Model Provider Adapter

```

```

```
七、产品一句话介绍
```

中文：

```
```text id="k5j3zt"
```

DecisionOS 是一个可私有化部署的 AI Agent 工作台，支持用户通过聊天式界面提交任务，AI 自动拆解、检索知识库、调用工具并实时展示执行过程，适用于代码仓库分析、日志诊断、文档处理和团队内部 AI 协作。

英文：

```
```text id="o30sq3" DecisionOS is a self-hosted AI agent workspace that lets teams submit tasks through a chat-like interface, watch agent execution in real time, connect private knowledge bases, call tools, and generate actionable results for code analysis, log diagnosis, document processing, and team collaboration.
```

```

```

```
八、最终判断
```

这个 AI 产品方向不是从零开始，而是从你已有 IM 系统自然升级：

```
```text id="bd9bf2"
```

IM 系统解决实时连接和消息推送
AI Task 解决任务抽象
Agent Runtime 解决执行过程
Tool Calling 解决能力扩展
RAG 解决私有知识
Workflow 解决复杂流程
Open Source Packaging 解决传播

真正有开源潜力的版本不是“大而全”，而是：

```text id="6v4v4m" 私有化部署 + 实时 Agent 执行流 + 代码仓库分析 / 日志诊断 两个强场景

第一版只要做到：

```text id="7bx48i"  
上传代码仓库
AI 生成执行计划
实时展示分析过程
调用工具读取文件
生成项目报告
支持一键 Docker 部署
README 漂亮清楚

就已经具备开源传播的基础。